

List 3

Heuristic Search and Planning

Artur D. Pescador

Universidade Federal do Rio Grande do Sul

Advanced Artificial Intelligence — 2026/1

1 Exercise 1

1.1 (a) Planning tasks in positive normal form

1.1.1 (i) Π_1 unsolvable, but Π_1^+ has an optimal plan of length 2

$$\begin{aligned} P &= \{a, b, c\}, & I &= \{a\}, & G &= \{c\} \\ o_1 : \text{pre} &= \{a\}, & \text{add} &= \{b\}, & \text{del} &= \{a\} \\ o_2 : \text{pre} &= \{a, b\}, & \text{add} &= \{c\}, & \text{del} &= \{\} \end{aligned}$$

In Π_1 (original): o_1 is the only applicable operator in $I = \{a\}$. After o_1 the state is $\{b\}$; o_2 requires a , which was deleted. No sequence reaches c . Π_1 **is unsolvable**.

In Π_1^+ (relaxation): $\{a\} \xrightarrow{o_1} \{a, b\} \xrightarrow{o_2} \{a, b, c\} \supseteq G$. A plan of length 2 is optimal (no plan of length 1 can add c from $\{a\}$). Therefore $h^+(\Pi_1) = 2$.

1.1.2 (ii) Π_2 with optimal plan of length 2, but Π_2^+ unsolvable: IMPOSSIBLE

Such a task **does not exist**. Any plan $\langle o_1, \dots, o_k \rangle$ that solves Π_2 also solves Π_2^+ : the relaxation only removes delete effects, so each operator o_i remains applicable (its preconditions are never removed) and still produces all its add effects. Therefore, $h^*(\Pi) < \infty \Rightarrow h^+(\Pi) < \infty$.

1.1.3 (iii) $O = \{o_1, o_2, o_3\}$ and Π_3^+ with optimal plan of length 4: IMPOSSIBLE

Such a task **does not exist** in classical STRIPS (without conditional effects). In the relaxed task the state is *monotone*: propositions are never deleted. Since the add effects of each operator are fixed (independent of the state), applying an operator a second time cannot add any new proposition — all of them are already present after the first application. Therefore, in any optimal relaxed plan each operator appears *at most once*, and the plan length is at most $|O| = 3 < 4$.

Note: with *conditional effects* the argument breaks down. A conditional effect of the form “when C : add e ” only fires if condition C holds at application time. Under the relaxation, only deletes are removed but conditional add effects remain, so the same operator applied

twice can yield different add effects (since C may have become true between applications), invalidating the bound $|O|$. Example with $O = \{o_1, o_2, o_3\}$, $I = \{\}$, $G = \{goal\}$:

$$\begin{aligned} o_1 : \text{pre} &= \{\}, \quad \text{add} = \{a\}, \quad \text{when } \{c\}: \text{add } \{b\} \\ o_2 : \text{pre} &= \{\}, \quad \text{add} = \{c\} \\ o_3 : \text{pre} &= \{a, b\}, \quad \text{add} = \{goal\} \end{aligned}$$

Optimal relaxed plan of length 4: o_1 (adds a), o_2 (adds c), o_1 again (now adds b), o_3 (adds $goal$). No plan of length 3 suffices, since b is only obtained after c is added by o_2 .

1.1.4 (iv) Infinite family: $h^*(P_i) = i + 1$ and $h^+(P_i) = 1$ for all $i \geq 1$

Construction of P_i .

$$\begin{aligned} P_i &= \{p, d_1, d_2, \dots, d_i, goal\}, \\ I_i &= \{p, d_1, d_2, \dots, d_i\}, \quad G_i = \{goal, d_1, d_2, \dots, d_i\}. \end{aligned}$$

Operators:

$$\begin{aligned} use_p : \text{pre} &= \{p\}, \quad \text{add} = \{goal\}, \quad \text{del} = \{p, d_1, \dots, d_i\} \\ restore_j (j = 1, \dots, i) : \text{pre} &= \{\}, \quad \text{add} = \{d_j\}, \quad \text{del} = \{\} \end{aligned}$$

Relaxation Π_i^+ : In I_i we apply use_p ; deletes are ignored, so the relaxed state becomes $\{p, d_1, \dots, d_i, goal\} \supseteq G_i$. $\mathbf{h}^+(\mathbf{P}_i) = \mathbf{1}$ for all i .

Original P_i : After use_p the state is $\{goal\}$ (all d_j deleted). Each $restore_j$ must be applied separately. Optimal plan: $use_p, restore_1, \dots, restore_i$, with length $i + 1$. $\mathbf{h}^*(\mathbf{P}_i) = \mathbf{i} + \mathbf{1}$, which grows without bound.

	$h^*(P_i)$	$h^+(P_i)$
$i = 1$	2	1
$i = 2$	3	1
$i = 3$	4	1
i general	$i + 1$	1

1.2 (b) Analysis of the Visitall instance

Instance. A line of 5 locations $x_0 - x_1 - x_2 - x_3 - x_4$. The robot starts at x_2 (already visited). Goal: $\{vis_{x_0}, vis_{x_1}, vis_{x_2}, vis_{x_3}, vis_{x_4}\}$.

Movement action:

$$m_{c \rightarrow n} : \quad \text{pre} = \{ar_{x_c}, c_{cn}\}, \quad \text{add} = \{ar_{x_n}, vis_{x_n}\}, \quad \text{del} = \{ar_{x_c}\}.$$

$h^*(I) = 6$. To visit $\{x_0, x_1, x_3, x_4\}$ starting from x_2 on a line, the optimal strategy goes all the way in one direction and then the other:

$$x_2 \rightarrow x_1 \rightarrow x_0 \quad (2 \text{ moves}) \quad x_0 \rightarrow x_1 \rightarrow x_2 \rightarrow x_3 \rightarrow x_4 \quad (4 \text{ moves}).$$

Total: $\mathbf{h}^*(\mathbf{I}) = \mathbf{6}$ (confirmed by $A^* + \text{LM-cut}$).

$h^+(I) = 4$. In the relaxed task ar_{x_x} is never deleted; the robot can be at multiple locations simultaneously. Starting from ar_{x_2} :

$$\begin{aligned} m_{x_2 \rightarrow x_1} &\rightarrow vis_{x_1}, & m_{x_2 \rightarrow x_3} &\rightarrow vis_{x_3}, \\ m_{x_1 \rightarrow x_0} &\rightarrow vis_{x_0}, & m_{x_3 \rightarrow x_4} &\rightarrow vis_{x_4}. \end{aligned}$$

Four parallel moves visit all 5 locations. $\mathbf{h}^+(\mathbf{I}) = 4$ (confirmed by A*+LM-cut on the relaxed task).

Relaxed Task Graph (RTG). Figure 1 shows the RTG (nodes c_{ij} , all with cost 0 and originating from I_s , are omitted for clarity).

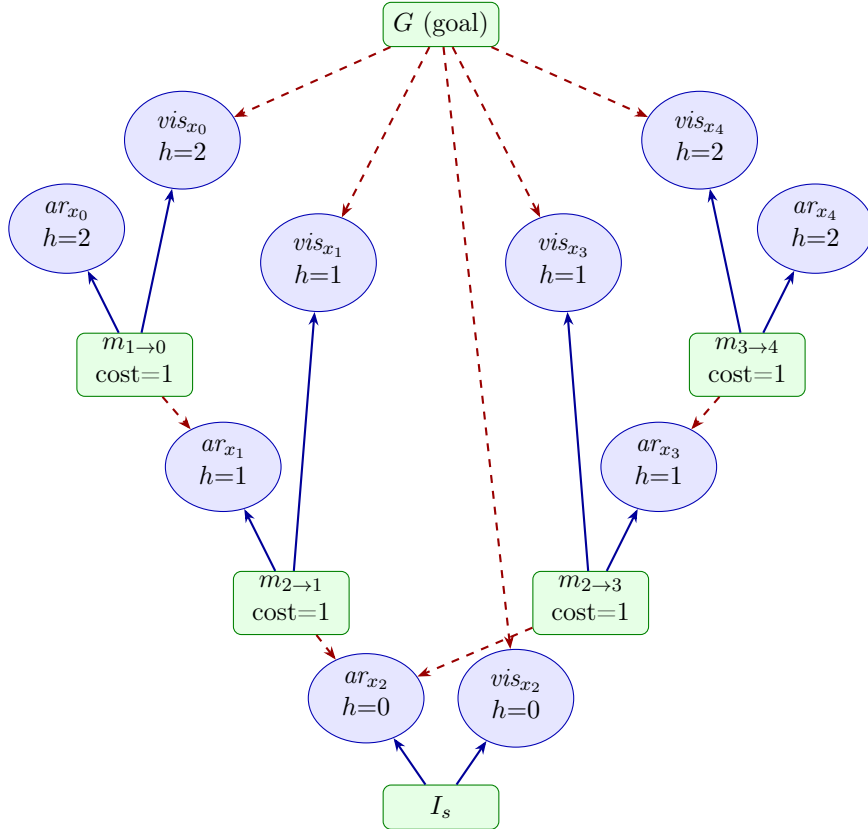


Figure 1: Relaxed Task Graph of the Visittall instance (5 locations in a line). Blue ovals = OR nodes (propositions); green rectangles = AND nodes (operators/initial node/goal node). Annotations “ $h=k$ ” indicate the h^{max} cost of each OR node. Nodes c_{ij} (cost 0, all originating from I_s) are omitted.

Computing $h^{max}(I)$. $h^{max}(p)$ = minimum over the *achievers* of p of:

$$\max(h^{max}(\text{preconditions})) + \text{operator cost}.$$

Derivation (left chain; symmetric on the right):

$$h^{max}(ar_{x_1}) = h^{max}(m_{x_2 \rightarrow x_1}) = \max(0, 0) + 1 = 1,$$

$$h^{max}(ar_{x_0}) = h^{max}(m_{x_1 \rightarrow x_0}) = \max(1, 0) + 1 = 2.$$

G is an AND node, so $h^{max}(G) = \max$ over the goal propositions:

$$\boxed{h^{max}(I) = \max(2, 1, 0, 1, 2) = 2.}$$

Table 1: h^{max} and h^{add} costs for each proposition in the Visitall instance.

Proposition	h^{max}	h^{add}
$ar_{x_{x_2}}, vis_{x_{x_2}}, c_{ij}$ (all)	0	0
$ar_{x_{x_1}}, vis_{x_{x_1}}, ar_{x_{x_3}}, vis_{x_{x_3}}$	1	1
$ar_{x_{x_0}}, vis_{x_{x_0}}, ar_{x_{x_4}}, vis_{x_{x_4}}$	2	2

Computing $h^{add}(I)$. $h^{add}(p)$ = minimum over the *achievers* of p of:

$$\sum(h^{add}(\text{preconditions})) + \text{operator cost.}$$

The individual costs coincide with h^{max} (Table 1) because each dependency chain is linear. G is an AND node, so $h^{add}(G) = \sum$ over the goal propositions:

$$h^{add}(I) = 2 + 1 + 0 + 1 + 2 = 6.$$

Summary for the Visitall instance.

$h^{max}(I)$	$h^+(I)$	$h^*(I)$	$h^{add}(I)$	$h^{FF}(I)$
2	4	6	6	4

Admissibility chain: $h^{max} \leq h^+ \leq h^* \leq h^{add} \Rightarrow 2 \leq 4 \leq 6 \leq 6. \checkmark$

$h^{add}(I) = h^*(I) = 6$: although inadmissible in general, h^{add} coincides with the optimum in this instance. $h^{FF}(I) = h^+(I) = 4$: the extracted relaxed plan is exactly the optimal relaxed plan of 4 actions $\{m_{x_2 \rightarrow x_1}, m_{x_2 \rightarrow x_3}, m_{x_1 \rightarrow x_0}, m_{x_3 \rightarrow x_4}\}$.

2 Exercise 2

2.1 (a) most_conservative_valuation (and_or_graph.cc)

The method implements the most conservative valuation without costs (generalised BFS). It starts with all AND nodes that have no successors and propagates to predecessors by incrementing `num_forced_successors`: an AND node is enqueued when it reaches its total number of successors; an OR node on the first occurrence.

Unit test output:

```
Verifying graph g1 ... Verification of g1 successful
Verifying graph g2 ... Verification of g2 successful
Verifying graph g3 ... Verification of g3 successful
```

2.2 (b) RelaxedTaskGraph and is_goal_relaxed_reachable

AND/OR graph structure:

- One **OR node** per proposition.
- **Initial AND node I_s** : no successors; edges $OR(p) \rightarrow I_s$ for $p \in I$.
- **Goal AND node G** : edges $G \rightarrow OR(g)$ for $g \in \mathcal{G}$.

- **One AND node per operator** with `direct_cost = cost(o)`, edges to preconditions and reverse edges from each effect. (One node per operator, not per effect, prevents h^{FF} from counting the cost multiple times.)

`is_goal_relaxed_reachable()` runs `most_conservative_valuation()` and returns `goal_node.forced_true`.

Experiments 2(b): Blind A* vs. A* with RTG in the Castle domain

Table 2: Comparison between blind search and RTG in the Castle domain. “ ∞ ” = no solution; T. = search time (s).

Instance	h^*	Blind (A*)		RTG (A*)		Obs.
		Exp.	T. [s]	Exp.	T. [s]	
castle-2-2-8	4	8	<0.001	8	<0.001	—
castle-3-3-8	9	84	<0.001	84	<0.001	—
castle-4-3-5	15	263	<0.001	263	0.002	—
castle-5-4-7	∞	12	<0.001	6	<0.001	RTG prunes dead ends
castle-5-4-9	19	7350	0.019	7350	0.118	—
castle-5-4-10	∞	0	<0.001	0	<0.001	Unsolvability in relaxation
castle-6-4-7	26	24146	0.054	24022	0.454	RTG 8× slower
castle-7-5-4	35	3680953	10.4	3659512	132.4	RTG 13× slower

Conclusion: RTG only reduces expansions when detecting *dead ends* (castle-5-4-7: 12→6; castle-5-4-10: detected immediately). The per-state computational cost is much higher (recomputes reachability at every state), making wall-clock time worse on larger instances.

2.3 (c) weighted_most_conservative_valuation and h^{add}

Cost-aware version: priority queue (*min-heap*) ordered by `additive_cost`.

- **OR node:** `cost = minach[additive_cost(ach) + direct_cost(OR)]`. The cheapest achiever is stored in `achiever` (used in (e)).
- **AND node:** `cost = sum` of successor costs plus `direct_cost` (replacing sum with max yields h^{max}).

2.4 (d) Experiments with h^{add} in the Castle domain

Search: `eager_greedy([planopt_add()])` (values identical to the built-in `add()` on all instances).

Observations:

- $h^{add}(I) \gg h^*$: inadmissible — sums costs per goal atom without discounting shared operators.
- Dramatically reduces expansions compared to blind search (castle-5-4-9: 29 vs. 7350), but may produce suboptimal plans (castle-5-4-9: plan cost 21 vs. $h^* = 19$).

Table 3: h^{add} in the Castle domain (greedy search). h^+ obtained with A*+LM-cut on the relaxed task. T. in ms.

Instance	$h^{add}(I)$	$h^+(I)$	h^*	Plan	Exp.	T. (ms)
castle-2-2-8	6	4	4	4	5	0.06
castle-3-3-8	53	9	9	9	10	0.56
castle-4-3-5	161	14	15	15	21	1.30
castle-5-4-9	232	19	19	21	29	5.30
castle-6-4-7	555	≥ 24	26	26	43	9.07

2.5 (e) ff_cost_of_goal: heuristic h^{FF} (relaxed_task_graph.cc)

h^{FF} extracts a *relaxed plan* from the *achievers* recorded in (c): starting from the goal node, AND nodes follow all successors and OR nodes follow only the stored *achiever*. Returns the sum of `direct_cost` of the collected nodes (equivalent to the sum of operator costs in the relaxed plan).

2.6 (f) Experiments with h^{FF} in the Castle domain

Search: `eager_greedy([planopt_ff()])`.

Table 4: Comparison between h^{add} and h^{FF} in the Castle domain (greedy search).

Instance	$h^{add}(I)$	$h^{FF}(I)$	$h^+(I)$	h^*	Plan (h^{add})	Plan (h^{FF})
castle-2-2-8	6	4	4	4	4	4
castle-3-3-8	53	9	9	9	9	9
castle-4-3-5	161	18	14	15	15	15
castle-5-4-9	232	25	19	19	21	19
castle-6-4-7	555	34	≥ 24	26	26	30

Table 5: Expansions and search time (ms) for all strategies.

Instance	Admissible A*		Greedy h^{add}		Greedy h^{FF}	
	Blind	RTG	Exp.	T. (ms)	Exp.	T. (ms)
castle-2-2-8	8	8	5	0.06	5	0.08
castle-3-3-8	84	84	10	0.56	10	0.60
castle-4-3-5	263	263	21	1.30	16	1.26
castle-5-4-9	7350	7350	29	5.30	20	4.63
castle-6-4-7	24146	24022	43	9.07	35	8.28

Observations:

- $h^{FF}(I) \ll h^{add}(I)$: each operator counted once vs. h^{add} accumulating per atom; h^{FF} much closer to h^+ (castle-5-4-9: $h^{FF} = 25$, $h^+ = 19$, $h^{add} = 232$).
- h^{FF} expands fewer states and finds better plans in most cases (castle-5-4-9: optimal plan 19 vs. 21 with h^{add}).
- Exception: castle-6-4-7, where h^{add} greedy finds the optimum (26) but h^{FF} greedy reaches 30 — greedy search does not guarantee optimality.